ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - TIN HỌC

Automatic Change-Point Detection in Time Series via Deep Learning

Giảng viên hướng dẫn: **Tô Đức Khánh**

Học viên thực hiện: Phạm Cung Lê Nhân Vũ – 25C01049
Nguyễn Thị Thanh Hoài – 25C24003
Nguyễn Nhật Quang – 25C23002

TP. HỒ CHÍ MINH, 04–2026

Mục lục

1 Giới thiệu **2**

2 Cơ Sở Lý Thuyết **3**
2.1 Ký hiệu Toán học 3
2.2 Bài toán Phát hiện Điểm thay đổi và Phép kiểm định CUSUM 3
2.3 Deep Neural Networks 4
2.4 Cơ sở Toán học: Rủi ro và Sai số Tổng quát hóa của DNN 6

3 Phương pháp luận **8**
3.1 Chuyển đổi bài toán: Từ Kiểm định Thống kê sang Phân loại Nhị phân 8
3.2 Cấu hình MLP và sự tương đương giữa DNN với CUSUM 8
3.3 Cấu hình Residual CNN cho chuỗi phức tạp 9
3.4 Tạo dữ liệu và Tiền xử lý 10
3.5 Huấn luyện và Tối ưu hóa 11
3.6 Thuật toán định vị đa điểm thay đổi (Algorithm 1) 11

4 Thực nghiệm và kết quả đánh giá **13**
4.1 Thí nghiệm 1: Synthetic data với một điểm thay đổi (Single Change-Point) 13
4.2 Thí nghiệm 2: Phát hiện hỗn hợp nhiều loại thay đổi phức tạp (trích từ bài báo gốc) 15
4.3 Thí nghiệm 3: Ứng dụng thực tế trên bộ dữ liệu HASC (Real-world Application) . . 15

5 Kết luận **16**

1 Giới thiệu

Trong kỷ nguyên dữ liệu lớn, việc theo dõi và phân tích các chuỗi dữ liệu thời gian (time series) là yêu cầu cốt lõi trong nhiều lĩnh vực khoa học và công nghiệp. Một trong những bài toán quan trọng nhất là phát hiện điểm thay đổi (Change-Point Detection). Điểm thay đổi là vị trí hoặc thời điểm mà tại đó, các thuộc tính thống kê cơ bản của dữ liệu (chẳng hạn như giá trị trung bình, phương sai, hoặc độ dốc) đột ngột thay đổi.

Bài toán này giúp trả lời hai câu hỏi chính: *Có sự thay đổi có cấu trúc nào xảy ra trong dữ liệu hay không? Và nếu có thì sự thay đổi đó xảy ra ở đâu, vào lúc nào?*. Việc tự động hóa việc nhận diện các thay đổi này giúp tạo ra các ứng dụng thực tiễn trong cuộc sống, ví dụ như trong xử lý tín hiệu, phát hiện sự biến đổi trong tế bào, dự báo khí hậu, theo dõi các hành vi chuyển động của con người, v.v...

Mặc dù có nhiều vai trò quan trọng, song các phương pháp truyền thống vẫn còn nhiều hạn chế, hầu hết nằm ở việc chúng không đủ tốt để xử lý các dữ liệu phức tạp hoặc không tuân theo các phân phối như Gaussian. Trong nhiều trường hợp, việc tự thiết kế một kiểm định thống kê phù hợp cũng đòi hỏi nhiều giả định và tinh chỉnh thủ công. Đây là một quy trình dài dòng, dễ xảy ra sai sót và phụ thuộc quá nhiều vào các giả định toán học cứng nhắc. Ví dụ, kiểm định CUSUM chỉ hoạt động tốt khi dữ liệu có nhiễu Gaussian độc lập. Khi áp dụng vào dữ liệu thực tế phức tạp chứa nhiễu tự tương quan (auto-correlated noise) hoặc nhiễu có phân phối đuôi dài (heavy-tailed Cauchy noise), các kiểm định tiêu chuẩn này thường xuyên thất bại hoặc hoạt động rất kém. Thậm chí, với các dạng dữ liệu quá phức tạp, có thể còn chưa tồn tại một thống kê kiểm định chuẩn mực nào.

Điều này tạo động lực để chúng ta tìm ra một phương pháp khác phù hợp hơn về mặt lý thuyết và thực tiễn. Thông qua các kết quả toán học, Li và đồng nghiệp chứng minh rằng Deep Neural Network (DNN) có thể đóng vai trò như một bộ kiểm định học từ dữ liệu: bài toán kiểm định thống kê được chuyển thành bài toán phân loại nhị phân, và mô hình được huấn luyện từ dữ liệu quá khứ hoặc dữ liệu synthetic. Khi dữ liệu huấn luyện đủ lớn, DNN có thể đạt hiệu năng ít nhất tương đương CUSUM trong trường hợp lý tưởng. Quan trọng hơn, nhờ khả năng tối ưu hóa trực tiếp trên dữ liệu, DNN có thể học được cơ chế phát hiện tốt hơn khi nhiễu có cấu trúc phức tạp hoặc vi phạm các giả định cổ điển.

Mục tiêu chính của đồ án này là hiện thực hóa, đánh giá và mở rộng phương pháp phát hiện điểm thay đổi tự động dựa trên Deep Learning được đề xuất bởi Li và đồng nghiệp [5], đồng thời đưa ra nhận xét thực nghiệm từ bản tái cài đặt. Bản tái cài đặt PyTorch của nhóm cũng được đối chiếu trực tiếp với mã nguồn TensorFlow chính thức của các tác giả [4]. Đồ án sẽ thực hiện các bước sau:

- Xây dựng và triển khai Multi Layer Perceptron (MLP) với độ rộng $4\log_2(n)$ để phát hiện single change-point, cùng kiến trúc Residual CNN (ResCNN) cho các kịch bản nhiễu phức tạp hơn.
- Huấn luyện mô hình với Binary Cross-Entropy loss, kết hợp các bước tiền xử lý như min-max normalization và data augmentation bằng reverse sequence để tăng kích thước tập huấn luyện.

2 Cơ Sở Lý Thuyết

2.1 Ký hiệu Toán học

Các ký hiệu toán học dưới đây cung cấp một chuẩn mực thống nhất sẽ được sử dụng xuyên suốt trong báo cáo, đặc biệt phục vụ cho việc xây dựng và chứng minh các định lý, bổ đề về giới hạn sai số ở các phần sau.

Ký hiệu	**Ý nghĩa**
$[n]$	Tập hợp các số nguyên dương từ 1 đến n, tức $\{1, \ldots, n\}$
$\mathbf{1}_n$	Véc-tơ cột có tất cả các thành phần là 1 và độ dài là n
$\mathbf{1}\{\cdot\}$	Hàm chỉ thị (Indicator function), trả về 1 nếu điều kiện đúng, ngược lại 0
$\lvert \cdot \rvert$	Độ lớn hoặc giá trị tuyệt đối
$\lVert x \rVert_p$	Chuẩn p (với $p \geq 1$) của véc-tơ $x = (x_1, \ldots, x_n)^\top$: $\quad \lVert x \rVert_p := \left(\sum_{i=1}^n \lvert x_i \rvert^p\right)^{1/p}$
$\lVert x \rVert_\infty$	Chuẩn vô cực của véc-tơ x: $\quad \lVert x \rVert_\infty := \max_i \lvert x_i \rvert$

Bảng 1: Bảng tra cứu các ký hiệu toán học cơ bản

2.2 Bài toán Phát hiện Điểm thay đổi và Phép kiểm định CUSUM

1. Định nghĩa bài toán

Phát hiện điểm thay đổi (Change Point Detection) là bài toán tìm kiếm vị trí mà tại đó các thuộc tính thống kê của một chuỗi dữ liệu bị thay đổi. Một trong những bài toán cơ bản nhất là phát hiện sự thay đổi về giá trị trung bình (change in mean). Cụ thể, xét một chuỗi quan sát $X = (X_1, X_2, \ldots, X_n)^\top$. Bài toán đặt ra là kiểm định giả thuyết:

- H_0 (Không có thay đổi): X_i có cùng kỳ vọng μ với mọi $i = 1, \ldots, n$.
- H_1 (Có một điểm thay đổi tại τ): Tồn tại vị trí $\tau \in [n-1]$ sao cho X_i có kỳ vọng μ_L với $i \leq \tau$, và μ_R với $i > \tau$ ($\mu_L \neq \mu_R$).

2. Phép kiểm định CUSUM

Để giải quyết bài toán trên, thống kê CUSUM (Cumulative Sum) truyền thống tính toán khoảng cách giữa trung bình của hai đoạn dữ liệu được chia cắt bởi vị trí k. Thống kê CUSUM được định nghĩa là:

$$T_n = \max_{1 \leq k < n} \sqrt{\frac{k(n-k)}{n}} |\bar{X}_{1:k} - \bar{X}_{k+1:n}| \tag{1}$$

trong đó $\bar{X}_{1:k}$ và $\bar{X}_{k+1:n}$ lần lượt là trung bình mẫu của các đoạn dữ liệu trước và sau k. Nếu T_n vượt qua một ngưỡng λ cho trước, ta bác bỏ giả thuyết H_0 và kết luận có điểm thay đổi. Vị trí đạt cực đại của T_n chính là ước lượng của điểm thay đổi $\hat{\tau}$.

3. Nhược điểm của CUSUM

Mặc dù là công cụ chuẩn mực trong lý thuyết thống kê, phương pháp CUSUM bộc lộ nhiều điểm yếu chí mạng khi đối mặt với dữ liệu thực tế:

- **Phụ thuộc giả định phân phối:** CUSUM đòi hỏi nhiễu của dữ liệu phải tuân theo phân phối chuẩn (Gaussian) và độc lập cùng phân phối (i.i.d).
- **Kém hiệu quả với dữ liệu phức tạp:** Khi áp dụng vào dữ liệu có nhiễu đuôi nặng (heavy-tailed) hoặc nhiễu tự tương quan (auto-correlated), CUSUM thường xuyên thất bại (báo động giả cao hoặc bỏ lọt điểm thay đổi).
- Ngoài ra, việc thiết kế thủ công các thống kê kiểm định tối ưu cho dữ liệu đa chiều hoặc tín hiệu phức tạp là cực kỳ khó khăn.

Những hạn chế này chính là động lực để nghiên cứu tiếp cận bài toán bằng học máy hoặc học sâu, cho phép mô hình học các đặc trưng phức tạp trực tiếp từ dữ liệu thay vì dựa vào các giả định toán học cứng nhắc.

2.3 Deep Neural Networks

Thay vì thiết kế các thống kê thủ công như CUSUM, nghiên cứu sử dụng Deep Neural Network (DNN) để đóng vai trò như một bộ phân loại trực tiếp trên tín hiệu đầu vào.

1. Cấu trúc Multi Layer Perceptron (MLP)

Gọi $L \in \mathbb{Z}^+$ là số lượng lớp ẩn và $m = (m_1, \ldots, m_L)^\top$ là vector độ rộng các lớp ẩn, tức m_i là số unit trong lớp ẩn thứ i. Với một mạng có L lớp ẩn, ta quy ước: $m_0 = n$ (kích thước đầu vào) và $m_{L+1} = 1$ (đầu ra nhị phân).

Với bất kỳ vector bias $b = (b_1, \ldots, b_r)^\top \in \mathbb{R}^r$, định nghĩa **hàm kích hoạt có dịch chuyển** (shifted activation function) $\sigma_b : \mathbb{R}^r \to \mathbb{R}^r$ như sau:

$$\sigma_b((y_1, \ldots, y_r)^\top) = (\sigma(y_1 - b_1), \ldots, \sigma(y_r - b_r))^\top$$

trong đó $\sigma(x)$ là một hàm kích hoạt phi tuyến bất kỳ (ví dụ: ReLU, Sigmoid,...) làm nhiệm vụ biến đổi đặc trưng.

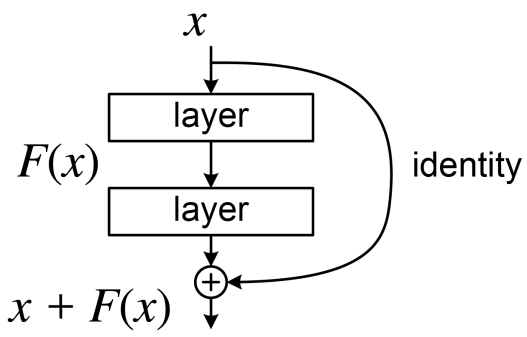
Mô hình MLP được biểu diễn toán học bởi một hàm hợp (composite function) $h : \mathbb{R}^n \to \{0, 1\}$ dưới dạng:

$$h(x) := \sigma_\lambda^* \Big(W_L \sigma_{b_L} \big(W_{L-1} \sigma_{b_{L-1}} (\cdots W_1 \sigma_{b_1} (W_0 x) \cdots) \big) \Big) \tag{2}$$

Trong đó:

- $\sigma_\lambda^*(x) = \mathbf{1}\{x > \lambda\}$ với $\lambda > 0$. Lớp đầu ra sử dụng hàm bước Heaviside dịch chuyển này để đưa ra quyết định phân loại nhị phân.
- $W_\ell \in \mathbb{R}^{m_{\ell+1} \times m_\ell}$ là ma trận trọng số của lớp ℓ.

Lớp hàm $\mathcal{H}_{L,m}$ được định nghĩa là tập tất cả các hàm $h(x)$ có L lớp ẩn và vector độ rộng lớp ẩn m.



Hình 1: Minh họa cấu trúc Multi Layer Perceptron (MLP)

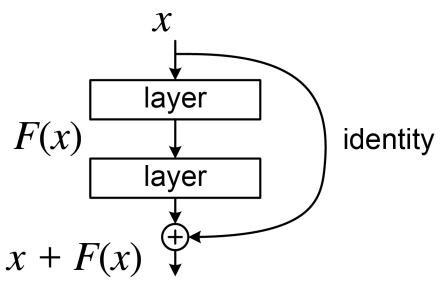
2. Residual CNN (ResCNN)

Đối với các dữ liệu chuỗi thời gian có độ phức tạp cao, cấu trúc MLP cơ bản tỏ ra không đủ để trích xuất đặc trưng. Mạng ResCNN [1] được áp dụng với thành phần cốt lõi là các khối Residual (Residual blocks) và kết nối tắt (skip connections).

Về mặt toán học, thay vì yêu cầu các lớp mạng học trực tiếp một hàm ánh xạ mong muốn $\mathcal{H}(x)$, khối Residual định nghĩa một hàm phần dư $\mathcal{F}(x) := \mathcal{H}(x) - x$ và tối ưu hóa nó. Đầu ra của khối được tính bằng cách cộng trực tiếp đầu vào x thông qua kết nối tắt:

$$y = \mathcal{F}(x, \{W_i\}) + x \tag{3}$$

trong đó $\mathcal{F}(x, \{W_i\})$ đại diện cho phần dư cần học và x là danh tính (identity mapping).



Hình 2: Minh họa cấu trúc của một khối Residual (Residual Block)

Ưu điểm: Cơ chế cộng trực tiếp này tạo ra một "đường cao tốc"cho luồng thông tin, giúp giải quyết triệt để vấn đề đạo hàm bị triệt tiêu (vanishing gradient) khi mạng trở nên quá sâu. Nhờ đó, mô hình cực kỳ phù hợp để khai phá các cấu trúc ẩn phức tạp trong tín hiệu cảm biến hoặc các loại nhiễu phi tuyến.
Nhược điểm: Sự gia tăng độ phức tạp dẫn đến tiêu tốn năng lực tính toán và bộ nhớ cao, đồng thời cần tập dữ liệu đủ lớn để tránh hiện tượng quá khớp (overfitting).

3. Các Hàm mất mát (Loss Functions)

a) Hàm mất mát 0-1 (0-1 Loss): Về mặt lý thuyết, nghiên cứu dùng hàm mất mát 0-1 để đánh giá trực tiếp tỷ lệ phân loại đúng sai. Việc này cho phép áp dụng lý thuyết chiều VC (Vapnik-Chervonenkis dimension) để thiết lập các giới hạn chặn trên chặt chẽ cho sai số tổng quát hóa.

b) Hàm mất mát Binary Cross-Entropy (BCE Loss):
Tuy nhiên, hàm 0-1 có đạo hàm bằng 0 ở hầu hết mọi nơi, khiến thuật toán Gradient Descent không thể hoạt động. Do đó, trong thực hành huấn luyện, DNN thường sử dụng Binary Cross-Entropy (BCE) - một hàm chuẩn mực cho các bài toán phân loại nhị phân. Cho một dự đoán $\hat{y} \in [0, 1]$ và nhãn thực tế $y \in \{0, 1\}$, hàm BCE đối với một tập dữ liệu kích thước N được định nghĩa:

$$L_{BCE}(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \tag{4}$$

Tính chất toán học và Ứng dụng: BCE Loss xuất phát từ lý thuyết Thông tin (Information Theory) và tương đương với việc cực đại hóa hàm ước lượng hợp lý cực đại (Maximum Likelihood Estimation - MLE) cho phân phối Bernoulli. Nó phạt cực kỳ nặng (tiến tới vô cực) khi dự đoán sai hoàn toàn với độ tự tin cao. BCE cực kỳ phù hợp cho các bài toán phát hiện bất thường hay điểm thay đổi, nơi hệ thống không chỉ cần đoán đúng nhãn mà còn phải xuất ra được xác suất chính xác nhằm thiết lập các ngưỡng báo động (threshold) trong thực tế.

2.4 Cơ sở Toán học: Rủi ro và Sai số Tổng quát hóa của DNN

Tại sao cần phân tích toán học? Việc ứng dụng Deep Learning vào thống kê thường vấp phải chỉ trích vì tính chất "hộp đen"(black-box), gây khó khăn cho việc diễn giải cơ chế ra quyết định. Phần lý thuyết dưới đây cung cấp nền tảng toán học cho cách tiếp cận này, bằng cách chứng minh rằng một MLP đơn giản có thể mô phỏng chính xác thuật toán CUSUM. Do đó, hiệu năng của DNN được đặt trên một cơ sở lý thuyết rõ ràng, thay vì chỉ là quan sát thực nghiệm.

1. Nền tảng: Phân tích Rủi ro của CUSUM

Trước khi đánh giá DNN, rủi ro phân loại sai của CUSUM cần được thiết lập chặt chẽ. Xét chuỗi dữ liệu quan sát

$$X \sim \mathcal{N}_n(\mu, I_n)$$

độ dài n, trong đó vector trung bình

$$\mu = (\mu_L, \ldots, \mu_L, \mu_R, \ldots, \mu_R)^\top$$

có điểm thay đổi (nếu có) tại vị trí $\tau \in [n-1]$. Định nghĩa $\eta := \tau/n \in (0, 1)$ là tỷ lệ vị trí điểm thay đổi.

Bổ đề 4.1 kiểm soát hai loại sai số cơ bản của CUSUM với mức ý nghĩa ϵ:

- **Kiểm soát Báo động giả (Type I Error):** Khi không có sự thay đổi ($\mu_L = \mu_R$), xác suất thống kê CUSUM vượt quá ngưỡng $\sqrt{2\log(n/\epsilon)}$ được chặn trên bởi ϵ.
- **Kiểm soát Bỏ lọt (Type II Error):** Khi có sự thay đổi với cường độ tín hiệu đủ mạnh $|\mu_L - \mu_R|\sqrt{\eta(1-\eta)} > \sqrt{8\log(n/\epsilon)/n}$, xác suất bỏ lọt cũng được chặn bởi ϵ.

Để đánh giá tổng thể, định nghĩa không gian tham số $\Theta(B)$ đại diện cho các phân phối hoặc là không có thay đổi, hoặc có điểm thay đổi với Tỷ lệ Tín hiệu trên Nhiễu (SNR) tối thiểu là B. Theo **Hệ quả 4.1**, với ngưỡng $\lambda = B\sqrt{n}/2$, rủi ro phân loại sai của CUSUM giảm theo hàm mũ đối với tín hiệu B:

$$\mathbb{P}(h_\lambda^{\text{CUSUM}}(X) \neq Y) \leq ne^{-nB^2/8}$$

2. Giới hạn đối với MLP một lớp ẩn (Định lý 4.2)

Một MLP 1 lớp ẩn với $2n-2$ unit đã được chứng minh là đủ để mô phỏng hoàn hảo phép biến đổi CUSUM. Với tập huấn luyện kích thước N, sai số tổng quát hóa của bộ Tối thiểu hóa rủi ro thực nghiệm (ERM) bằng MLP với xác suất $1-\delta$ được chặn bởi:

$$\mathbb{P}(h_{\text{ERM}}(X) \neq Y|\mathcal{D}) \leq ne^{-nB^2/8} + C\sqrt{\frac{n^2\log(n)\log(N)+\log(1/\delta)}{N}}$$

Giới hạn này chỉ ra rằng sai số của MLP hội tụ về sai số gốc của CUSUM khi tập dữ liệu huấn luyện N đủ lớn ($N \gg n^2 \log n$).

3. Tối ưu hóa kiến trúc với MLP nhiều lớp (Định lý 4.3)

Các unit biểu diễn CUSUM thường có độ tương quan rất cao. Bằng cách "tỉa" (pruning) các unit dư thừa, mạng có thể mô phỏng CUSUM chỉ với $O(\log n)$ unit. Áp dụng cho MLP nhiều lớp ẩn (L lớp), **Định lý 4.3** nới lỏng đáng kể yêu cầu về dữ liệu:

$$\mathbb{P}(h_{\text{ERM}}(X) \neq Y|\mathcal{D}) \leq 2\lfloor\log_2(n)\rfloor e^{-nB^2/24} + C\sqrt{\frac{L^2 n\log^2(Ln)\log(N)+\log(1/\delta)}{N}}$$

Việc sử dụng cấu trúc đa lớp hẹp ($4\lfloor\log_2(n)\rfloor$ unit) giúp giảm rủi ro quá khớp (overfitting), cho phép thuật toán đạt độ chính xác cao ngay cả khi mẫu N tương đối nhỏ.

3 Phương pháp luận

Bài báo [5] tiếp cận bài toán phát hiện điểm thay đổi bằng cách chuyển đổi nó thành một bài toán phân loại (classification) sử dụng Deep Neural Network (DNN). Thay vì thiết kế một thống kê kiểm định thủ công dựa trên mô hình dữ liệu (như phương pháp CUSUM truyền thống [6]), phương pháp này học trực tiếp một hàm ánh xạ $\psi(X)$ từ tập dữ liệu huấn luyện để quyết định xem một chuỗi thời gian X có chứa điểm thay đổi hay không.

3.1 Chuyển đổi bài toán: Từ Kiểm định Thống kê sang Phân loại Nhị phân

Ý tưởng then chốt là xem mỗi chuỗi quan sát $X = (X_1, \ldots, X_n)^\top$ như một *mẫu dữ liệu* (sample) và gán cho nó một *nhãn* (label) $Y \in \{0, 1\}$:

- $Y = 0$: Chuỗi X không chứa điểm thay đổi (trạng thái tĩnh - pure state).
- $Y = 1$: Chuỗi X chứa ít nhất một điểm thay đổi (trạng thái chuyển tiếp - transitional state).

Bằng cách này, toàn bộ bài toán kiểm định giả thuyết H_0 vs H_1 (đã định nghĩa ở Mục 2.2) được quy về bài toán tối ưu hóa một bộ phân loại nhị phân $\psi : \mathbb{R}^n \to \{0, 1\}$ sao cho rủi ro phân loại sai $\mathbb{P}(\psi(X) \neq Y)$ là nhỏ nhất. Khác với thống kê cổ điển mà ở đó ta kiểm soát riêng rẽ sai lầm Loại I và sai lầm Loại II, phương pháp này tối ưu trực tiếp **tỷ lệ phân loại sai tổng thể** (0-1 loss / mis-classification error rate) trên toàn bộ không gian tham số.

3.2 Cấu hình MLP và sự tương đương giữa DNN với CUSUM

Dựa trên kiến trúc MLP tổng quát đã trình bày ở Mục 2.3.1, phần này giải thích cách lựa chọn cấu hình cụ thể cho bài toán phát hiện điểm thay đổi, và đặc biệt là chứng minh rằng MLP có thể **bao hàm hoàn toàn** phương pháp CUSUM.

1. Tại sao cần đúng $2n - 2$ unit?

Một trong những kết quả lý thuyết quan trọng nhất của bài báo là chứng minh rằng thống kê CUSUM có thể được biểu diễn *chính xác* bằng một MLP 1 lớp ẩn với $2n - 2$ ReLU unit.

Cụ thể, nhắc lại rằng thống kê CUSUM tại vị trí k là:

$$T_n(k) = \sqrt{\frac{k(n-k)}{n}} \left| \bar{X}_{1:k} - \bar{X}_{k+1:n} \right|$$

Biểu thức bên trong dấu giá trị tuyệt đối là một hàm **tuyến tính** của X, tức tồn tại một vector trọng số $w_k \in \mathbb{R}^n$ sao cho:

$$T_n(k) = |w_k^\top X|$$

Ta có thể phân tích giá trị tuyệt đối bằng hàm ReLU:

$$|z| = \max(z, 0) + \max(-z, 0) = \mathrm{ReLU}(z) + \mathrm{ReLU}(-z) \tag{5}$$

Do đó, mỗi vị trí $k \in \{1, \ldots, n-1\}$ cần đúng **2 ReLU unit**: một cho $\max(w_k^\top X, 0)$ và một cho $\max(-w_k^\top X, 0)$. Với $n-1$ vị trí cần kiểm tra, tổng cộng cần $2(n-1) = 2n-2$ hidden units.

Ý nghĩa: Kết quả này chứng minh rằng lớp hàm $\mathcal{H}_{1,(2n-2)}$ (MLP 1 lớp ẩn, $2n-2$ hidden units) đã đủ để *chứa* bộ phân loại CUSUM. Nói cách khác, khi huấn luyện một mạng MLP với kiến trúc này bằng phương pháp Tối thiểu hóa Rủi ro Thực nghiệm (ERM), bộ phân loại tối ưu tìm được sẽ **ít nhất tốt bằng** CUSUM và có tiềm năng **vượt trội** hơn nếu trong dữ liệu tồn tại các cấu trúc phức tạp mà CUSUM không nắm bắt được.

2. Pruning: Từ $O(n)$ xuống $O(\log n)$ hidden units

Mặc dù mạng $2n-2 = O(n)$ hidden units đảm bảo biểu diễn chính xác CUSUM, trong thực tế các unit này có **độ tương quan rất cao** với nhau (các trung bình $\bar{X}_{1:k}$ tại các vị trí k kề nhau gần như giống nhau). Bằng cách xấp xỉ, ta chỉ cần lấy mẫu ở $O(\log n)$ vị trí chia nhị phân (binary splitting) mà vẫn giữ được hầu hết thông tin của CUSUM.

Do đó, MLP nhiều lớp (L lớp ẩn) với chỉ $m = 4\lfloor \log_2(n) \rfloor$ hidden units mỗi lớp đã đủ mạnh giúp giảm đáng kể từ $O(n)$ xuống $O(\log n)$. Các tác giả thử nghiệm cấu hình $L \in \{1, 5, 10\}$ để đánh giá ảnh hưởng của độ sâu lên hiệu năng.

3. Biến đổi dữ liệu đầu vào (Pre-transformations)

Khi cần phát hiện các loại thay đổi khác ngoài trung bình mà không cần thiết kế mạng quá sâu, ta có thể biến đổi dữ liệu đầu vào trước khi đưa vào mạng. Cụ thể, bài báo sử dụng:

- **Bình phương:** Nối thêm x_t^2 vào đặc trưng đầu vào giúp mạng phát hiện sự thay đổi về *phương sai.*
- **Tích chéo:** Nối thêm $x_t \cdot x_{t+1}$ giúp mạng phát hiện sự thay đổi về *tự tương quan.*

Các biến đổi này được kích hoạt qua các tùy chọn cấu hình `use_squared` và `use_cross_product`, giúp tăng chiều đặc trưng đầu vào từ n lên tối đa $3n$ mà không cần thay đổi kiến trúc mạng.

3.3 Cấu hình Residual CNN cho chuỗi phức tạp

1. Tại sao dùng CNN?

Khi mở rộng bài toán từ việc chỉ phát hiện một loại thay đổi cơ bản (thay đổi giá trị trung bình) sang **phát hiện đồng thời nhiều loại thay đổi phức tạp** (ví dụ: hỗn hợp thay đổi về giá trị trung bình, phương sai và độ dốc), mạng MLP cơ bản không còn đủ khả năng xử lý. Các tác giả nhận thấy có sự tương đồng lớn giữa xử lý tín hiệu chuỗi thời gian và nhận dạng hình ảnh: cả hai đều cần trích xuất các *đặc trưng cục bộ* (local features) ở nhiều mức trừu tượng. Do đó, mạng CNN được sử dụng nhằm tự động học và trích xuất các đặc trưng đa dạng của nhiều loại điểm thay đổi khác nhau.

2. Tại sao dùng Residual blocks?

Mặc dù CNN rất tốt trong việc trích xuất đặc trưng, nếu chỉ đơn thuần xếp chồng (stacking) nhiều lớp CNN để làm mạng sâu hơn, mô hình sẽ gặp phải **hiện tượng triệt tiêu đạo hàm** (vanishing gradients) tức là các trọng số ở các lớp đầu tiên gần như không thể cập nhật được. Việc áp dụng kiến trúc khối Residual (đã trình bày ở Mục 2.3.2) với cơ chế $y = \mathcal{F}(x, \{W_i\}) + x$ là giải pháp cốt lõi: dòng đạo hàm có thể truyền ngược trực tiếp qua kết nối tắt, giúp mô hình học được các thay đổi phức tạp ngay cả khi mạng rất sâu.

3. Tại sao 21 blocks?

Con số 21 Residual blocks không phải được chọn ngẫu nhiên. Các tác giả đã tiến hành thử nghiệm với nhiều cấu trúc mạng khác nhau, thay đổi số lượng khối residual cũng như số lượng lớp fully connected trên các tập dữ liệu tự tạo. Kết quả cho thấy kiến trúc với **21 Residual blocks** tiếp nối bởi các lớp fully connected là điểm cân bằng tối ưu, vừa đủ sâu để bắt được các tín hiệu thay đổi hỗn hợp phức tạp, vừa không quá sâu để gây lãng phí tính toán hay overfitting.

4. Chi tiết kiến trúc

Kiến trúc cụ thể được sử dụng:

- **Backbone:** 21 Residual blocks với `base_channels` = 32, `kernel_size` = 8 cho mỗi lớp tích chập.
- **Dropout:** Tỷ lệ 0.3 được áp dụng nhằm giảm thiểu overfitting, đặc biệt quan trọng khi tập dữ liệu cảm biến (HASC) có kích thước hạn chế.
- **Classification Head:** Phần đầu ra sử dụng chuỗi các lớp kết nối đầy đủ (Dense layers). Tùy thuộc vào yêu cầu:
  - *Phân loại nhị phân* (có/không có điểm thay đổi): Lớp Dense cuối cùng có 1 nơ-ron + Sigmoid.
  - *Phân loại đa nhãn* (phân biệt loại thay đổi): Lớp Dense cuối cùng có số nơ-ron bằng số nhãn + Softmax.

3.4 Tạo dữ liệu và Tiền xử lý

1. Cấu trúc tập dữ liệu synthetic

Vì DNN cần lượng dữ liệu lớn để huấn luyện, nghiên cứu sử dụng dữ liệu synthetic. Tổng cộng N chuỗi thời gian độ dài n được sinh ra, chia thành hai nửa cân bằng:

- $N/2$ chuỗi có điểm thay đổi: $\mu_L = 0$, điểm thay đổi $\tau \sim \text{Unif}\{2, \ldots, n-2\}$, và cường độ tín hiệu μ_R được tính dựa trên công thức SNR:

$$b = \sqrt{\frac{8n \log(20n)}{\tau(n-\tau)}}$$

với $\mu_R \sim \text{Unif}([0.5b, 1.5b] \cup [-1.5b, -0.5b])$. Công thức này xuất phát trực tiếp từ điều kiện kiểm soát sai số Loại II trong Bổ đề 4.1 (Mục 2.4): để CUSUM có thể phát hiện được điểm thay đổi, cường độ tín hiệu $|\mu_L - \mu_R|$ phải vượt ngưỡng $\sqrt{8\log(n/\epsilon)/n}$ sau khi hiệu chỉnh theo vị trí τ. Như vậy, việc chọn μ_R theo công thức này đảm bảo synthetic data nằm đúng vùng *ranh giới phát hiện* của CUSUM — không quá dễ (tín hiệu quá mạnh) cũng không quá khó (tín hiệu quá yếu).

- $N/2$ chuỗi không có điểm thay đổi: $\mu_L = \mu_R = 0$.

Nghiên cứu thử nghiệm trên 4 kịch bản nhiễu:

- **S1:** Nhiễu Gaussian i.i.d $\xi_t \sim \mathcal{N}(0, 1)$.
- **S1':** Nhiễu AR(1) với hệ số tự tương quan cố định ρ: $\varepsilon_t = \rho \cdot \varepsilon_{t-1} + \xi_t$.
- **S2:** Nhiễu AR(1) với hệ số biến thiên theo thời gian $\rho_t \sim \text{Unif}([0, 1])$, $\xi_t \sim \mathcal{N}(0, 2)$.
- **S3:** Nhiễu Cauchy đuôi nặng $\xi_t \sim \text{Cauchy}(0, 0.3)$.

2. Tiền xử lý dữ liệu

- **Chuẩn hóa Min-Max:** Mỗi chuỗi được chuẩn hóa riêng biệt về khoảng $[0, 1]$ bằng phương pháp Min-Max Scaling. Điều này giúp bộ phân loại bất biến với các phép biến đổi affine (cộng hoặc nhân với hằng số). Tuy nhiên, đối với nhiễu đuôi nặng (kịch bản S3 - Cauchy), Min-Max sẽ thất bại do các giá trị cực đoan. Vì vậy, nghiên cứu thay thế bằng phương pháp **chuẩn hóa mạnh** sử dụng trung bình cắt tỉa (trimmed mean) và độ lệch chuẩn cắt tỉa (trimmed std) với tỷ lệ cắt 10% ở mỗi phía.
- **Tăng cường dữ liệu (Data Augmentation):** Sử dụng kỹ thuật đảo ngược thứ tự các phần tử trong chuỗi đầu vào. Vì chuỗi đảo ngược vẫn giữ nguyên nhãn phân loại (có hoặc không có điểm thay đổi), kỹ thuật này nhân đôi kích thước tập huấn luyện một cách hiệu quả.

3.5 Huấn luyện và Tối ưu hóa

Mặc dù mục tiêu lý thuyết là tối ưu hàm mất mát 0-1 (đã trình bày ở Mục 2.3.3a), trong thực tế hàm 0-1 không khả vi nên nghiên cứu huấn luyện mạng bằng cách cực tiểu hóa hàm **Binary Cross-Entropy with Logits Loss** (BCEWithLogitsLoss), kết hợp lớp Sigmoid và BCE Loss vào cùng một hàm để đảm bảo tính ổn định số học.

Các siêu tham số huấn luyện được sử dụng theo cấu hình thực nghiệm:

3.6 Thuật toán định vị đa điểm thay đổi (Algorithm 1)

DNN được huấn luyện để phân loại một đoạn dữ liệu kích thước cố định: *có* hoặc *không có* điểm thay đổi. Tuy nhiên, trong thực tế, ta cần một công cụ có khả năng ước lượng **vị trí chính xác** của *nhiều điểm thay đổi* trên một chuỗi thời gian dài, chưa được gán nhãn. Thuật toán 1 cung cấp một quy trình tổng quát để chuyển đổi bộ phân loại nhị phân thành một công cụ định vị đa điểm thay đổi.

Thuật toán hoạt động dựa trên cơ chế **cửa sổ trượt** (sliding window), lấy ý tưởng tương tự như thủ tục MOSUM (Moving Sum) trong thống kê cổ điển. Cụ thể, các bước tiến hành như sau:

Siêu tham số	**Giá trị**
Bộ tối ưu hóa (Optimizer)	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Tốc độ học (Learning rate)	0.001
Số epochs	200
Batch size	32
Tỷ lệ tập validation	10%
Early stopping (patience)	20 epochs
Weight decay (MLP)	0.0
Weight decay (ResCNN)	0.0001
Dropout (ResCNN)	0.3

Bảng 2: Bảng siêu tham số huấn luyện

1. **Dự đoán trên từng cửa sổ:** Trích xuất các phân đoạn dữ liệu liên tiếp có độ dài n (đóng vai trò là cửa sổ trượt), ký hiệu là $X^*_{[i,i+n)}$. Đưa từng phân đoạn qua bộ phân loại đã huấn luyện ψ để dự đoán xem cửa sổ đó có chứa điểm thay đổi hay không. Kết quả trả về cho mỗi cửa sổ là nhãn $L_i \in \{0, 1\}$.

2. **Tính trung bình trượt:** Tính giá trị trung bình trượt (rolling average) của các nhãn qua các cửa sổ chồng lấn (overlapping):

$$\bar{L}_i = \frac{1}{n} \sum_{j=i-n+1}^{i} L_j$$

   Bước này làm mịn tín hiệu và giảm ảnh hưởng của nhiễu phân loại sai lẻ tẻ.

3. **Nhận diện vùng thay đổi:** Xác định các phân đoạn cực đại (maximal segments) mà ở đó xác suất trung bình $\bar{L}_i \geq \gamma$, với ngưỡng γ thường được chọn là 1/2. Mỗi phân đoạn này tương ứng với một vùng nghi ngờ chứa điểm thay đổi.

4. **Định vị chính xác:** Ước lượng vị trí chính xác của từng điểm thay đổi $\hat{\tau}_r$ bằng cách tìm thời điểm mà $\bar{L}_i$ đạt giá trị cao nhất bên trong mỗi phân đoạn vừa xác định ở Bước 3.

Đầu vào và Đầu ra:

- **Đầu vào:** Chuỗi dữ liệu mới chưa biết điểm thay đổi $(x^*_1, \ldots, x^*_{n^*})$, bộ phân loại đã huấn luyện ψ, và giá trị ngưỡng γ.
- **Đầu ra:** Danh sách các vị trí thời gian của những điểm thay đổi đã được ước lượng $(\hat{\tau}_1, \ldots, \hat{\tau}_{\hat{\nu}})$.

Trong thực tế, các tác giả đã áp dụng thành công thuật toán kết hợp cửa sổ trượt này trên bộ dữ liệu HASC (Human Activity Sensing Consortium) để quét qua toàn bộ chuỗi tín hiệu cảm biến, qua đó nhận diện và định vị chính xác vị trí thời gian của nhiều điểm thay đổi khác nhau trong chuỗi hoạt động thể chất liên tục của con người.

4 Thực nghiệm và kết quả đánh giá

Phần này trình bày kết quả đánh giá các mô hình DNN được cài đặt lại, so sánh trực tiếp với các phương pháp thống kê truyền thống trên cả synthetic data và dữ liệu HASC.

4.1 Thí nghiệm 1: Synthetic data với một điểm thay đổi (Single Change-Point)

Mục tiêu của thí nghiệm này là kiểm chứng lý thuyết mà tác giả đưa ra: liệu DNN có thể hoạt động như một "nhà thống kê tự động"khi đối mặt với các loại nhiễu dữ liệu khác nhau hay không.

Thiết lập thí nghiệm: Thuật toán sinh dữ liệu tạo ra chuỗi độ dài $n = 100$ với 4 kịch bản nhiễu nền:

- **S1:** Nhiễu Gaussian độc lập $\mathcal{N}(0, 1)$, không có tự tương quan ($\rho = 0$).
- **S1':** Nhiễu AR(1) với hệ số tự tương quan cố định $\rho = 0.7$.
- **S2:** Nhiễu AR(1) biến đổi theo thời gian, với $\rho_t \sim \text{Unif}[0, 1]$.
- **S3:** Nhiễu phân phối đuôi nặng Cauchy (Cauchy noise) với tỉ lệ scale = 0.3.

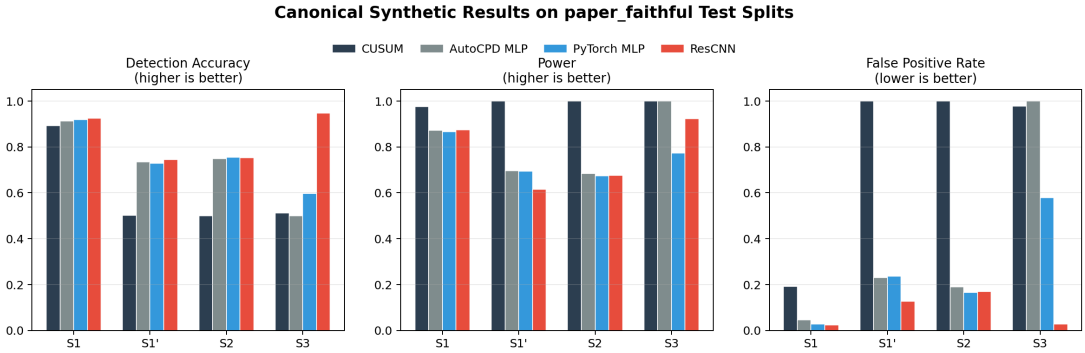
Ba cách trình bày kết quả được sử dụng để bảo đảm phần thực nghiệm nhất quán giữa hình minh hoạ và bảng số:

- **Tóm tắt trực quan chính (Hình 3):** độ chính xác phát hiện, Power và FPR được vẽ trực tiếp từ các file `eval_results.json` của bốn phương pháp: PyTorch MLP, ResCNN, AutoCPD MLP và CUSUM trên cùng split `paper_faithful_test`.
- **Bảng so sánh chính thức (Bảng 3):** các mô hình được huấn luyện trên $N_{\text{train}} = 10000$ chuỗi cố định và đánh giá trên tập kiểm thử $N_{\text{test}} = 2000$ chuỗi paper-faithful, sinh deterministic từ `data/paper_faithful/` với hash SHA-256 lưu sẵn.
- **Bảng đối chiếu tham chiếu (Bảng 4):** MLP TensorFlow/Keras của tác giả được huấn luyện và đánh giá trên đúng các split canonical nói trên để kiểm tra tính nhất quán với bản tái cài đặt PyTorch.

Quá trình huấn luyện sử dụng Adam [3], learning rate 0.001 và Binary Cross-Entropy loss. Multi Layer Perceptron (MLP, $h = 4\lfloor \log_2 n \rfloor = 24$ hidden units) và Residual CNN (21 Residual blocks) được so sánh trực tiếp với kiểm định CUSUM truyền thống.

Bảng 3: So sánh độ chính xác phát hiện trên 4 kịch bản nhiễu (ngưỡng quyết định 0.5, $N_{\text{train}} = 10000$, $N_{\text{test}} = 2000$). Dữ liệu sinh deterministic từ `data/paper_faithful/`, hash SHA-256 để đảm bảo tính integrity được ghi trong `artifacts/synthetic/manifest.json`.

Kịch bản	**NN MLP**	**NN ResCNN**	**CUSUM**
S1 (Gauss iid)	0.9190	**0.9255**	0.8915
S1' (AR(1), $\rho = 0.7$)	0.7290	**0.7435**	0.5005
S2 (AR(1), $\rho_t \sim \text{Unif}[0, 1]$)	**0.7540**	0.7525	0.5000
S3 (Cauchy, scale=0.3)	0.5970	**0.9475**	0.5110



Hình 3: Tóm tắt kết quả canonical trên bốn kịch bản nhiễu từ đúng các artifact đang nuôi bảng trong report: Detection Accuracy, Power và False Positive Rate của CUSUM, AutoCPD MLP, PyTorch MLP và ResCNN trên các split `paper_faithful_test`.

Bảng 4: So sánh trực tiếp với MLP AutoCPD trên cùng split `paper_faithful`.

Kịch bản	**AutoCPD MLP**	**NN MLP**	**NN ResCNN**
S1 (Gauss iid)	0.9125	0.9190	**0.9255**
S1' (AR(1), $\rho = 0.7$)	0.7335	0.7290	**0.7435**
S2 (AR(1), $\rho_t \sim$ Unif$[0, 1]$)	0.7475	**0.7540**	0.7525
S3 (Cauchy, scale=0.3)	0.5000	0.5970	**0.9475**

Kết quả trên dữ liệu chuẩn (Nhiễu Gaussian, S1): Đúng như lý thuyết trong bài báo dự đoán, khi dữ liệu chỉ chứa nhiễu Gaussian đơn giản, cả MLP và ResCNN đều đạt hiệu suất bám sát kiểm định CUSUM (Hình 3). Trong Bảng 3, ResCNN đạt độ chính xác cao nhất (**0.9255**), MLP đạt 0.9190, và bản thân CUSUM cũng đạt 0.8915 và tất cả đều ở mức xấp xỉ tiệm cận. Điều này xác nhận DNN có khả năng **tái lập** công thức thống kê cổ điển khi dữ liệu phù hợp với giả định của nó.

Đối chiếu trực tiếp với mã nguồn tham chiếu AutoCPD: Khác với các lần kiểm tra trước đây trên shared test set riêng trong thư mục `comparison/`, ở bản hiện tại nhóm đã huấn luyện trực tiếp MLP TensorFlow/Keras của tác giả (`https://github.com/Jieli12/AutoCPD`) trên đúng các split canonical `paper_faithful` của repo. Kết quả ở Bảng 4 cho thấy AutoCPD MLP bám rất sát bản PyTorch MLP trên S1, S1' và S2 (0.9125 so với 0.9190 ở S1, 0.7335 so với 0.7290 ở S1', và 0.7475 so với 0.7540 ở S2). Điều này củng cố rằng bản tái cài đặt PyTorch của nhóm là nhất quán với hướng tiếp cận gốc của tác giả ở các kịch bản Gaussian và AR.

Kết quả trên dữ liệu phức tạp (AR(1) và Cauchy): Khi giả định của CUSUM bị vi phạm, lợi thế của DNN trở nên rõ rệt:

- Trên **S1' và S2** (nhiễu tự tương quan), CUSUM sụp đổ về xác suất ngẫu nhiên (~ 0.50 với FPR ≈ 1.0, gần như luôn báo có thay đổi). MLP và ResCNN giữ được độ chính xác trên 0.72.
- Trên **S3** (nhiễu Cauchy đuôi nặng), sự khác biệt kiến trúc trở nên cực kỳ rõ. AutoCPD MLP sụp về mức ngẫu nhiên đúng nghĩa (0.5000 với FPR = 1.0), MLP PyTorch chỉ đạt 0.5970, trong khi ResCNN sâu đạt tới **0.9475** và vượt CUSUM gần ~ 44 điểm phần trăm. So sánh trực tiếp này cho thấy giới hạn không chỉ nằm ở framework cài đặt, mà nằm ở chính lựa chọn MLP nông và tiền xử lý Min-Max khi gặp nhiễu đuôi nặng. Đây chính là minh chứng thực nghiệm cho luận điểm trung tâm của bài báo: trên các phân phối nhiễu mà thống kê cổ điển không có công thức tối ưu, các mô hình học sâu mới phát huy hết sức mạnh. (Theo phần supplementary mà bài báo gốc dẫn chiếu [5], một biến thể neural network có thêm bước truncation ở đầu vào còn vượt cả kiểm định phi tham số Wilcoxon trong bối cảnh nhiễu đuôi nặng; nhóm chưa thực hiện đối chiếu bổ sung đó.)

4.2 Thí nghiệm 2: Phát hiện hỗn hợp nhiều loại thay đổi phức tạp (trích từ bài báo gốc)

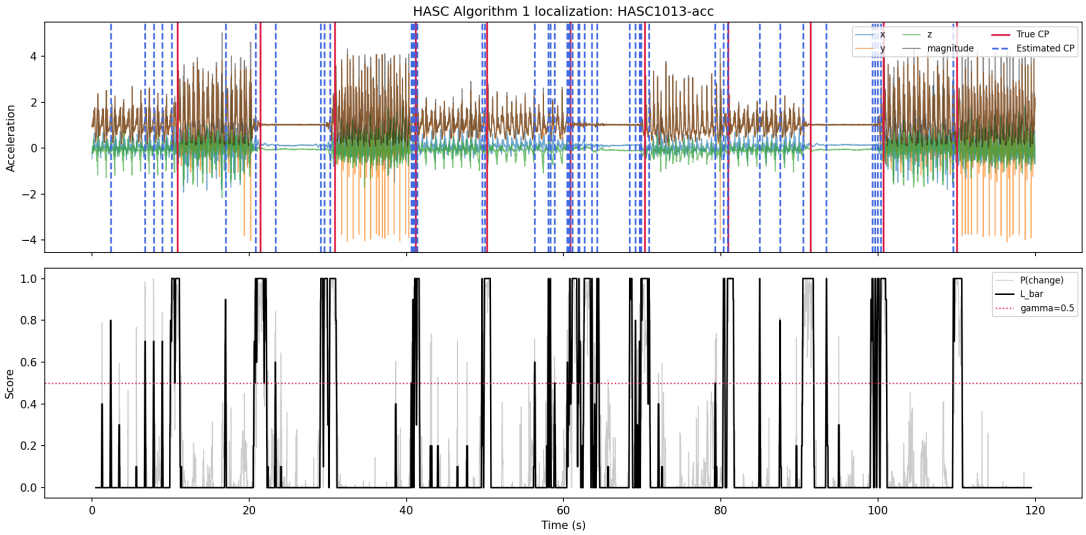
Trong thực tế, sự thay đổi không chỉ nằm ở giá trị trung bình. Bài báo gốc [5] thiết kế một thí nghiệm bổ sung trên tập dữ liệu gồm 10 000 chuỗi chứa **hỗn hợp các thay đổi về giá trị trung bình, phương sai và độ dốc**, đánh giá ResCNN cùng hai phương pháp thống kê mạnh: *Oracle likelihood* (bộ phân loại lý tưởng, đã biết trước loại thay đổi cần tìm) và *Adaptive likelihood* (kết hợp nhiều thống kê kiểm định).

Lưu ý phạm vi: Thí nghiệm này *không nằm trong phạm vi tái cài đặt* của nhóm, chúng em tóm lược kết quả từ bài báo gốc nhằm cho thấy bức tranh đầy đủ về khả năng của kiến trúc ResCNN. Theo các tác giả, ResCNN đạt độ chính xác phân loại **tương đương Adaptive ở tín hiệu yếu và vượt trội ở tín hiệu mạnh**, củng cố lập luận rằng mạng CNN sâu có thể tự động trích xuất đặc trưng cho các loại thay đổi đa dạng mà không cần thiết kế thống kê thủ công cho từng trường hợp.

4.3 Thí nghiệm 3: Ứng dụng thực tế trên bộ dữ liệu HASC (Real-world Application)

Để kiểm tra khả năng chạy trên real-world data, chúng em áp dụng mô hình ResCNN lên bộ dữ liệu **HASC (Human Activity Sensing Consortium)** [2].

Thiết lập và kết quả: HASC được dùng như một kiểm tra định tính trên dữ liệu cảm biến thật, không phải benchmark chính thức như phần synthetic. Repo hiện tại huấn luyện ResCNN nhị phân từ kênh độ lớn gia tốc $\sqrt{x^2 + y^2 + z^2}$, với cửa sổ trượt kích thước $n = 100$ và stride 10. Lần chạy hiện tại đọc được 18 recording có nhãn, tạo 3 854 cửa sổ cân bằng cho train/validation, và đạt best validation accuracy **0.8831**. Khi đưa checkpoint này vào Thuật toán 1, recording `HASC1013-acc.csv` có 11 ranh giới hoạt động theo label và mô hình ước lượng 47 điểm thay đổi. Kết quả này cho thấy pipeline đã chạy được end-to-end trên dữ liệu thật, nhưng mô hình hiện còn báo nhiều điểm thay đổi hơn label, do đó HASC ở đây chỉ nên được đọc như minh họa khả năng triển khai.



Hình 4: Minh họa định vị điểm thay đổi trên recording `HASC1013-acc.csv`. Đường đỏ là ranh giới hoạt động lấy từ file label; đường xanh nét đứt là điểm thay đổi do Thuật toán 1 ước lượng từ ResCNN. Panel dưới hiển thị xác suất cửa sổ có thay đổi, tín hiệu làm mượt $\bar{L}$ và ngưỡng $\gamma = 0.5$.

5 Kết luận

Đồ án đã hiện thực hóa và đánh giá lại phương pháp phát hiện điểm thay đổi tự động của Li và đồng nghiệp [5] theo hướng cài đặt lại sử dụng PyTorch và kiểm chứng bằng pipeline thực nghiệm có thể lặp lại. Trên phương diện lý thuyết, bài báo gốc cho thấy bài toán kiểm định thay đổi có thể được chuyển về một bài toán phân loại nhị phân học từ synthetic data mà trong đó MLP có thể tái biểu diễn được thống kê CUSUM [6], còn các kiến trúc sâu hơn như Residual CNN [1] có khả năng mở rộng sang các lớp nhiễu mà giả định cổ điển không còn phù hợp. Kết quả thực nghiệm trong repo hiện tại xác nhận lại luận điểm này trên bốn kịch bản synthetic chuẩn hóa như paper và có thể tái thiết lập để đảm bảo integrity của thí nghiệm.

Đóng góp của đồ án

- **Dùng PyTorch để tái cài đặt bài toán synthetic:** chúng em đã cài đặt lại hai họ mô hình chính của bài báo, gồm MLP và ResCNN, cùng quy trình huấn luyện với Binary Cross-Entropy và bộ tối ưu Adam [3]. Pipeline dữ liệu synthetic được chuẩn hóa lại thành các tập `paper_faithful` cố định, giúp việc huấn luyện và đánh giá không còn phụ thuộc vào dữ liệu sinh ngẫu nhiên ở mỗi lần chạy.
- **Thiết lập quy trình reproducible cho synthetic data:** toàn bộ bốn kịch bản S1, S1', S2 và S3 được đóng gói thành các tệp `.npz` canonical, đi kèm hash kiểm tra integrity và pipeline một lệnh để sinh dữ liệu, huấn luyện, đánh giá và tổng hợp kết quả. Nhờ đó, các bảng kết quả synthetic trong đồ án có thể được tái lập nhất quán từ cùng một mã nguồn.
- **Xác nhận lại vai trò của DNN trên dữ liệu đơn giản:** trên kịch bản S1 với nhiễu Gaussian độc lập, cả hai mô hình học đều bám sát baseline thống kê cổ điển. Cụ thể, MLP đạt độ chính xác 0.9190, ResCNN đạt 0.9255, trong khi CUSUM đạt 0.8915. Kết quả này phù hợp với lập luận của bài báo rằng DNN có thể học lại quy luật ra quyết định vốn do kiểm định CUSUM biểu diễn trong trường hợp dữ liệu đơn giản.
- **Cho thấy ưu thế rõ rệt trên các phân phối nhiễu khó:** khi chuyển sang các kịch bản có tự tương quan hoặc đuôi nặng, lợi thế của mô hình học trở nên rõ ràng hơn. Trên S1', ResCNN đạt 0.7435 và MLP đạt 0.7290, cao hơn rõ rệt so với CUSUM ở mức 0.5005. Trên S2, hai mô hình học gần như ngang nhau (0.7540 cho MLP và 0.7525 cho ResCNN), trong khi CUSUM vẫn chỉ ở mức 0.5000. Đáng chú ý nhất là S3 với nhiễu Cauchy, nơi ResCNN đạt tới 0.9475, vượt xa MLP (0.5970) và CUSUM (0.5110).
- **Củng cố luận điểm trung tâm của bài báo gốc:** các kết quả trên cho thấy Deep Learning không chỉ tái lập được kiểm định cổ điển trong điều kiện lý tưởng, mà còn có thể khai thác đặc trưng hiệu quả hơn khi dữ liệu vi phạm mạnh các giả định tham số. Trong số các kiến trúc đã khảo sát, ResCNN là mô hình mạnh nhất về tổng thể và đặc biệt vượt trội trên bài toán có nhiễu đuôi nặng.
- **Mở rộng kiểm tra định tính sang dữ liệu HASC:** ngoài phần synthetic chính, repo hiện tại đã có pipeline huấn luyện ResCNN trên tín hiệu gia tốc HASC và vẽ minh họa định vị bằng Thuật toán 1. Lần chạy hiện tại đạt best validation accuracy 0.8831 trên tập cửa sổ HASC cân bằng, đồng thời sinh được hình minh họa từ checkpoint `output/rescnn_hasc/best_model.pt`. Kết quả này chứng minh pipeline có thể chạy trên dữ liệu cảm biến thật, nhưng chưa được xem là benchmark định lượng chính.

Hạn chế và hướng phát triển

Mặc dù đã đạt được một pipeline synthetic tương đối hoàn chỉnh, đồ án vẫn còn một số giới hạn cần được mở rộng trong các bước tiếp theo:

- **Đánh giá chính thức hiện vẫn tập trung vào fixed-window classification:** kết quả reproducible hiện tại chủ yếu chốt ở bài toán phân loại cửa sổ có hay không có điểm thay đổi. Phần định vị trên chuỗi dài mới dừng ở mức minh họa, chưa được tổng hợp thành một bảng định lượng lớn với sai số $|\hat{\tau} - \tau|$ trên nhiều chuỗi.
- **Baseline thống kê còn chưa phong phú:** trong đồ án hiện tại, đối chứng chính vẫn là CUSUM. Việc bổ sung thêm các baseline phi tham số hoặc các phương pháp chuyên biệt cho nhiễu tự tương quan và nhiễu đuôi nặng sẽ cho cái nhìn đầy đủ hơn về mức độ vượt trội của ResCNN.
- **Chưa phân tích sâu yếu tố kiến trúc và tiền xử lý:** các kết quả hiện tại đã cho thấy MLP và ResCNN có hành vi khác nhau trên từng loại nhiễu, nhưng vẫn chưa có ablation study riêng cho các lựa chọn như pre-transform trên dữ liệu AR, chuẩn hóa robust cho nhiễu nặng đuôi, hoặc độ sâu tối ưu của ResCNN.
- **Phần HASC mới dừng ở mức kiểm tra định tính:** pipeline hiện tại chưa có split train/test theo người dùng và chưa có bảng định lượng sai số định vị trên toàn bộ recording. Trên `HASC1013-acc.csv`, mô hình ước lượng 47 điểm thay đổi trong khi label có 11 ranh giới hoạt động, cho thấy cần hiệu chỉnh thêm ngưỡng, smoothing và bước gộp điểm thay đổi trước khi dùng HASC như một benchmark chính thức.

Tổng kết

Nhìn chung, phần thực nghiệm synthetic trong repo hiện tại đã xác nhận được thông điệp quan trọng nhất của bài báo gốc: Deep Learning có thể đóng vai trò như một cơ chế phát hiện điểm thay đổi tự động, vừa tái lập được hành vi của kiểm định cổ điển trong trường hợp chuẩn, vừa thích nghi tốt hơn khi dữ liệu trở nên phức tạp. Việc xây dựng được một pipeline reproducible với dữ liệu canonical, kết quả có thể kiểm chứng lại và bảng tổng hợp nhất quán giúp đồ án không chỉ dừng ở mức tái cài đặt, mà còn đạt được giá trị phương pháp luận rõ ràng về tính lặp lại thực nghiệm. Trong bức tranh đó, ResCNN nổi lên là kiến trúc mạnh nhất, đặc biệt trên kịch bản Cauchy S3, và là hướng tiếp cận hứa hẹn nhất cho các nghiên cứu mở rộng sau này. Phần HASC bổ sung thêm bằng chứng kỹ thuật rằng cùng một hạ tầng mã nguồn có thể mở rộng sang tín hiệu thật, nhưng thông điệp chính của đồ án vẫn nên được đọc dựa trên các kết quả synthetic đã được chuẩn hóa và tái lập đầy đủ.

Tài liệu

[1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[2] Nobuo Kawaguchi, Nobuhiro Ogawa, Yohei Iwasaki, Katsuhiko Kaji, Tsutomu Terada, Kazuya Murao, Sozo Inoue, Yoshihiro Kawahara, Yasuyuki Sumi, and Nobuhiko Nishio. HASC challenge: Gathering large scale human activity corpus for the real-world activity understandings. In *Proceedings of the 2nd Augmented Human International Conference (AH '11)*, 2011.

[3] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015.

[4] Jie Li. AutoCPD: Reference implementation. `https://github.com/Jieli12/AutoCPD`, 2024. Accessed: 2026-04.

[5] Jie Li, Paul Fearnhead, Piotr Fryzlewicz, and Tengyao Wang. Automatic change-point detection in time series via deep learning. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 86(2):273–285, 2024.

[6] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.